scientific reports

OPEN

Parallel adaptive large neighborhood search based on spark to solve VRPTW

Songzuo Liu[1,2], Jian Sun[2]✉, Xiaohong Duan[1] & Guofang Liu[1]

Aiming at the multi-objective vehicle path planning problem with time windows (VRPTW), a Spark-based parallel Adaptive Large Neighborhood Search algorithm (Spark-ALNS) is proposed to solve it. The main design of the 4-point strategy: (1) Design a new simulated annealing algorithm cooling strategy to achieve a better jump out of the local optimal solution. (2) Adopt CW initialization to accelerate the convergence speed. (3) Use three destruction operators and three repair operators to implement local path optimization. (4) A new parallel strategy is proposed to improve the algorithm's accuracy and reduce the running time. To illustrate the algorithm's effectiveness, the arithmetic example in Solomon is used as an example. The experimental results show that the proposed Spark-ALNS can find better solutions, get the known optimal solutions for 41 out of 56 instances, and find new optimal solutions for 31 algorithms, which outperforms other evolutionary algorithms. The runtime is 3–5 times better than other parallel algorithms and is able to solve VRPTW effectively.

Keywords Adaptive large neighborhood search algorithm, Cooling strategy, Parallel algorithm, Spark, VRPTW

The Vehicle Routing Problem (VRP) minimizes logistics and distribution costs by studying the order in which distribution vehicles effectively arrange vehicle visits to customers when performing pickup and delivery services, to achieve optimal configuration of distribution vehicles and customers, and to minimize logistics and distribution costs under the premise of satisfying customer service requirements[1,2]. In new retail logistics and Intelligent vehicle[3–5], rapidly satisfying customer demand is critical to enhancing competitiveness. As a result, distribution routes must be optimized, and on-time delivery must be ensured to reduce costs and improve service quality. In order to deal with the problem, the Vehicle Routing Problem with Time Windows Constraints (VRPTW) was first studied in 1986, which Solomon[6]gave a more complete problem description and mathematical model of the VRPTW. Compared to the VRP problem, the VRPTW problem requires additional consideration of time window constraints, and its solution process is more complicated[7,8]. Therefore, how to effectively construct an optimization model for the VRPTW problem and solve it efficiently is the key problem to be solved in this paper.

To solve these problems, there is a huge amount of literature on this issue proposed to deal with VRP and its variants. Hosseinabadi AAR et al. proposed the Gravitational simulation-based on local search algorithm (GELS) to simulate the laws of gravity in the physical world and update the mass of the solution based on the gravitational force and velocity to solve the vehicle path problem (CVRP)[9]. Balas VE et al. developed a Hybrid Genetic Algorithm (HGA) for solving G-CARP, combining heuristic and simulated annealing algorithms to generate an initial solution, and then using a Genetic Algorithm (GA) to generate the best possible solution[10]. Nategh MN et al. proposed a meta-heuristic algorithm based on the Ant Colony Algorithm (ACO) combined with mutation operation and joining a 2-opt algorithm for solving VRP[11]. Tirkolaee EB et al. proposed the CLRPHTW problem and solved it based on the Variable Neighborhood Search (VNS) algorithm and Simulated Annealing (SA) algorithm[12]. Tirkolaee EB et al. proposed an new solution based on the improved Max-Min Ant System (IMMAS) to solve the multi-trip capacity-constrained arc routing problem (CARP) in the CPLEX solver[13]. Gocken T et al. used clustering initial populations, then constructed feasible paths for each cluster, and finally used a genetic algorithm to optimize the feasible paths to solve VRPTW[14]. Olaniyi OS et al. discuss modifications carried out on the Reproduction, Crossover and Mutation operators used in Genetic Algorithm which are applied in that order to the current population with a view to solving VRPTWSD[15]. Pirozmand P et al. decomposition of the original PFA algorithm into four sub-algorithms and discretization and coupling of

[1]Faculty of Information Engineering, Shandong Huayu University of Technology, Dezhou 253034, China. [2]College of Computer Science and Engineering, North Minzu University, Yinchuan 750021, China. ✉email: 13384514880@163.com

each sub-algorithm to form a new algorithm D-PFA for solving TSP[16].Tadaros M et al. combine Ant Colony Clustering and Variable Neighborhood Descent (VND) local search for solving the Hierarchical Multi-Exchange Multi-Level Vehicle Path Problem with Service Time Constraints (HMSME-VRP-ST)[17]. These algorithms have areas that need to be improved, such as learning the multi-objective cheetah optimizer (MOCO) proposed by GG Tejani, et al[18–20]. or making parallelization improvements, etc.

The traditional heuristic algorithms to solve VRPTW have the disadvantages of falling into the local optimum, slow convergence, and poor quality of the solution, so the Large Neighborhood Search (LNS) is proposed[21], which is based on the neighborhood of the solution, and searches for the neighborhood of the solution from the initial solution, and then repeat the process until the optimal solution is obtained. The main influence factor of neighborhood search is the size of the neighborhood. Although searching for larger neighborhoods can take a more longer time, relatively large neighborhoods can prevent the algorithm from falling into local optimization, thus giving better results. However, to obtain the best results in a reasonable amount of time, each algorithm needs to determine its neighborhood size individually. Ropke and Pisinger introduced the Adaptive Large Neighborhood Search (ALNS) algorithm based on the LNS algorithm[22]. The main difference between the two algorithms is that the ALNS algorithm selects one operator from the pool of destructive operators and one from the pool of restorative operators according to the weights in each iteration to manipulate the current set of paths[23]. In contrast, the LNS uses only one destructive operator and one restorative operator[24]. Due to introducing the adaptive layer, the algorithm is intelligent in selecting operators. ALNS is essentially a competition strategy between different neighborhoods to generate a feasible search operators solution. Furthermore, in each iteration, the destructive and repair operators work together to generate a new solution, defining the destructive neighborhood and the repair neighborhood, respectively, each containing a variety of operation operators.

To solve the problems of low solution quality and slow convergence of the ALNS algorithm, parallel programming is used for accelerated optimization. Distributed computing platforms can provide excellent parallel computing capabilities, such as Hadoop[25], Spark[26],and so on. Hadoop relies on the MapReduce framework, but the computation requires frequent data interactions between the CPU and the hard disk, which affects the computational efficiency and is unsuitable for iterative computation. Spark is an open-source platform that can be applied to large-scale data processing, and has the advantages of fast operating speed and easy programming, which makes Spark more suitable for iterative computation. Spark is an open-source platform that can be applied to large-scale data processing and has the advantages of fast speed and easy programming, which makes Spark more suitable for iterative computation. Algorithms that require several iterations run better on the Spark platform than on distributed platforms such as Hadoop[27], so using the Spark computing framework is a more suitable choice for iterative computation.

To solve the existing problem, in this paper, we propose Spark-based Parallel Adaptive Large Neighborhood Search (Spark-ALNS), which improves the cooling function, adopts CW initialization to speed up convergence, three destruction operators and three repair operators to increase the solution space, and optimizes the parallel approach. The algorithm's feasibility is verified by using the standard Solomon dataset as experimental data.

The main contributions of this paper are: (I) Aiming at the simulated annealing algorithm where both too high and too low temperatures affect the quality of the solution, the cooling function is improved to jump out the local optimum solution in a better way by tempering. (II) Given the problem that ALNS adopts random initialization, which leads to slow convergence of the solution, CW is used to generate a better initial solution. (III) For the problem that ALNS has a small neighborhood size and falls into local optimality, three destruction operators and three repair operators are used to increase the neighborhood size and the chance of jumping out of the local optimum. (IV) Aiming at the problem that the advantages of the Greedy Insertion operator in ALNS parallel algorithm can find the solution with high quality, but the time complexity is large, the parallel strategy is improved so that when the Greedy Insertion operator is selected, the Master will Shuffle according to the vehicle to accelerate the convergence speed.

Review of related works

Spark

Spark is a memory-based parallel computing framework for big data. The framework is based on memory computing, which improves real-time data processing in big data environments and ensures high fault tolerance and scalability. It allows users to deploy Spark on many common hardware to form clusters[28], it uses the dependencies of the Resilient Distributed Dataset (RDD) to construct the Directed Acyclic Graph (DAG) and decomposes the DAG into multiple stages for scheduling and processing[29].

The Spark cluster is divided into master nodes and worker nodes. The master node coordinates and schedules all worker nodes in the cluster. in each worker node, there are many executors are used to manage worker threads[30]. When an algorithm is run on Spark, a thread pool consisting of these worker threads is formed on all the worker nodes, and the inputs to the algorithm are partitioned and assigned to threads. Finally, the master node in Spark allocates the number of threads in the thread pool to execute the algorithm in parallel, and each thread carries a portion of the input to execute the algorithm. Similar to Genetic Algorithm[31,32], Ant Colony Algorithm[33,34], and Simulated Annealing Algorithm[35] that have many iterations will be stored directly in memory using Spark, avoiding the storage of intermediate results, greatly reducing disk I/O processing, and improving the efficiency of the iterations.

VRPTW Mathematical model

In order to clearly describe the mathematical model of VRPTW[36], the parameters used in the model are defined as follows:

V={0,1,……,n}: The collection of all customer points, the 0 is the distribution center.
K={1,……,n}: A collection of all delivery vehicles.
d_{ij}: The distance between customer i and customer j.
W_k: The maximum load capacity of vehicle j.
r_i: The demand for goods from customer i.
p_k: Vehicle k Collection of customer points served.
$[T_{ei},T_{li}]$: Time window constraints for customer i.
T_{ai}: The time when the vehicle arrives at customer point i.
T_{ui}: The waiting time of the vehicle at customer point i.
T_{si}: The service hours at customer i.

Based on the problem description and parameter definition of VRPTW, the following mathematical model is obtained:

$$x_{ijk} = \begin{cases} 1, \text{vehicle k goes from node i to node j} \\ 0, eles \end{cases} \tag{1}$$

$$y_{ik} = \begin{cases} 1, \text{vehicle k serves node i} \\ 0, eles \end{cases} \tag{2}$$

$$\mathrm{C}_1 = \min \sum_{i\in \mathrm{V}} \sum_{j\in \mathrm{V}} \sum_{k\in \mathrm{V}} x_{ijk} d_{ij} \tag{3}$$

$$C_2 = \min \sum_{j\in \mathrm{V}} \sum_{k\in \mathrm{K}} x_{0jk} \tag{4}$$

$$\sum_{\mathrm{i}\in \mathrm{V}} \sum_{j\in V} x_{ijk} = 1, \forall k \in K \tag{5}$$

$$\sum_{i\in V} r_i y_{ik} \leqslant W_k \tag{6}$$

$$\sum_{\mathrm{i}\in V} y_{\mathrm{ik}} = 1, \forall k \in K \tag{7}$$

$$\sum_{j\in V} x_{ijk} = |p_k| - 1, \forall k \in K \tag{8}$$

$$T_{ei} < T_{wi} + T_{ai} + T_{si} < T_{li}, \forall i \in V \tag{9}$$

$$T_{wi} = \max\{0, T_{ei} - T_{ai}\}, \forall i \in V \tag{10}$$

Where, Eqs. (1) and (2) for the decision variables, Eq. (3) is the minimum distribution distance of the objective function, Eq. (4) to the minimum number of vehicles used in the objective function, Eq. (5) that each vehicle can only serve one customer at a time, Eq. (6) that the vehicle's cargo load can not exceed the maximum load capacity, Eq. (7) that each customer can only be served by a vehicle, Eq. (8) that each vehicle can only form a distribution path, Eq. (9) indicates that the vehicle can only complete the delivery service within the service time window requested by the customer, Eq. (10) is the waiting service time of the vehicle i.

Parallel ALNS algorithm

The flow of the classical ALNS algorithm is divided into four main parts: constructing the initial feasible solution, selecting the destructed and repaired current solution, calculating the quality of the new solution, and updating the destructed and repaired operator scores and weights. The algorithm adaptively selects the removal and insertion operators that have performed well in the past through a roulette mechanism[37–39]. The generated new neighborhood solution satisfies the acceptance criterion based on simulated annealing, and then it can be used as the new current solution for the next iteration.

Initialization solution

High-quality initial solutions can effectively improve the convergence speed of the ALNS algorithm. The saving algorithm (CW) method is widely used in the vehicle path planning problem where the number of delivery vehicles enabled is unknown, and there are two main improvement methods, serial and parallel, in solving the VRPTW where the optimization objective is to minimize the transport distance or the total cost. Clarke refers to the triangular trilateral relationship when designing the algorithm, and the algorithm decides the order of execution by the degree of cost reduction or distance reduction brought about by merging the road sections. A superior initial solution can be generated using the saved journey method[40].

Destroy operator

The symbol S is used to denote the current solution, C denotes the customer that will be removed, RL denotes the set consisting of the removed customers, and P denotes the number of removed customers. Firstly, the

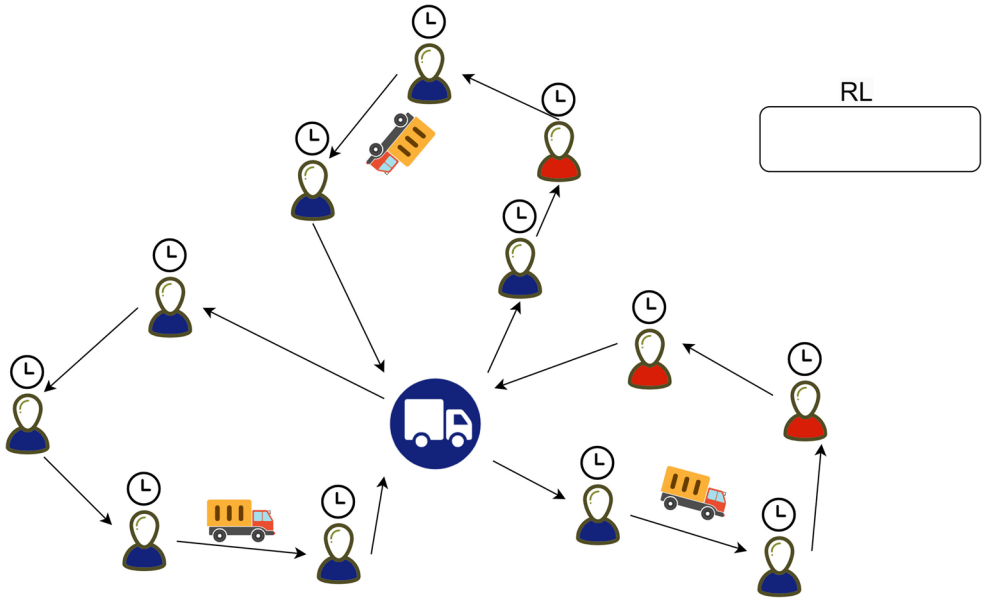

Figure 1. Selective destroy. In the complete distribution path. The customers to be removed are selected according to different destroy operators. The red customers in the figure are the customers selected for removal.

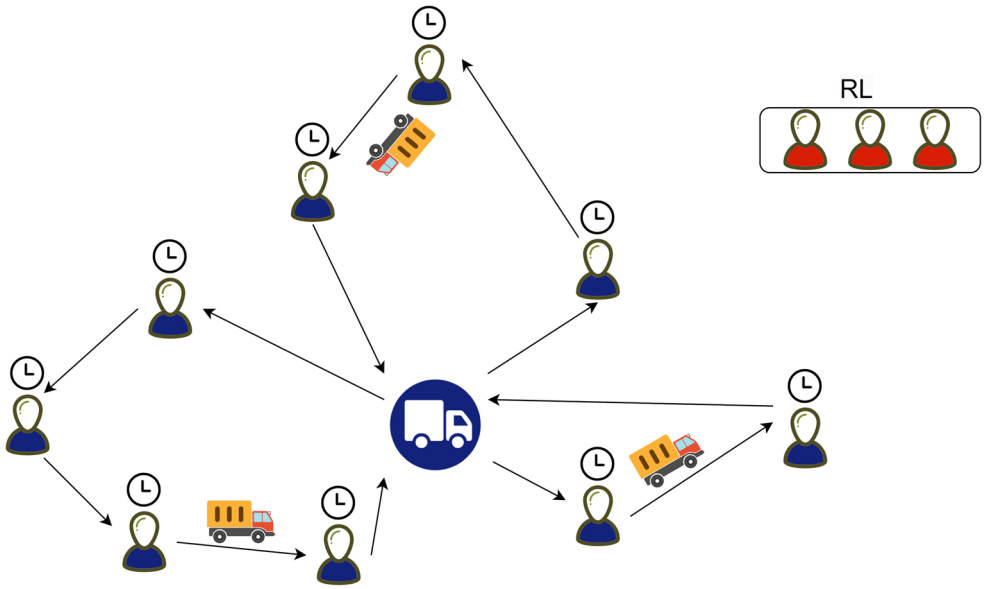

Figure 2. Destroyed. The clients selected for removal are removed from the original path and placed into the RL collection. The remaining clients generate new paths waiting to be repaired.

destruction operator is selected based on roulette, and 3 customers (Red marks) are selected for destruction as shown in Fig. 1. Secondly, as shown in Fig. 2, the selected 3 customers are put into the *RL* set, where the original path becomes a new distribution path waiting to be repaired. The three sabotage operators in this paper are as follows.

Random removal

Selecting a path in the set of paths, and selecting many merchants or customers from the original planning scheme randomly for that path to move into the set of customers to be inserted, and tries to insert other paths in the repair phase. For Random Removal, there is no order of selection. Although the selection of this operator

may lead to the generation of low-quality solutions, to some extent, it prevents the algorithm from falling into local optimization.

Route removal
Route Removal randomly selects a route from the current solution and removes all its clients. If the number of customers in the current route is less than *P*, another route is randomly selected and deleted. The process will be executed repeatedly until the total number of customers removed will reach or exceeds *P*. The method can effectively reduce the number of vehicles in use by deleting one vehicle and adding the customers served by this vehicle to other vehicles.

Related removal
Related Removal removes a random customer from *S* into *RL* as the first element of the set *RL*. The remaining *P-1* elements are selected as follows: randomly select one customer *X* at a time from the set *RL*, and arrange the remaining customers in *S* in descending order of relevance with *X*. The client *Y* with the highest correlation with *X* is selected from *S*. Remove *Y* from *S* and add it to *RL*. Repeat the process times until all *P* elements are selected. Where correlation is defined as:

$$R(i,j) = \frac{1}{C_{ij} + y_{ij}} \tag{11}$$

where C_{ij} denotes the normalizing value of *i* and *j* distances, the value of C_{ij}*is* between in [0,1], y_{ij} indicates whether *i* and *j* are on the same path or not.

Repair operator

As shown in Fig. 3, the repair operator is selected according to the roulette wheel, and the customers in the *RL* set are reinserted into the new location according to the relevant repair principle and finally the new distribution path is generated, and the three repair operators in this paper are as follows:

Random repair
The random insertion operator inserts clients from *RL* into the current solution in a random order. The time window constraint cannot be violated during the insertion process, and this operator is also set to maintain the diversity of the solutions.

Greedy insertion
Based on the idea of greedy strategy, the total transport cost of a customer in *RL* is recorded after its insertion at a possible location on a random path, and the P_{best} of each customer is determined in the order of its magnitude. The order of repair is determined according to the magnitude of the corresponding transport cost of each customer, and then that customer is removed from the *RL*. If there is no customer in the *RL* that needs the service, the repair phase is exited. In the contrary, the study continues with the insertion of the next customer.

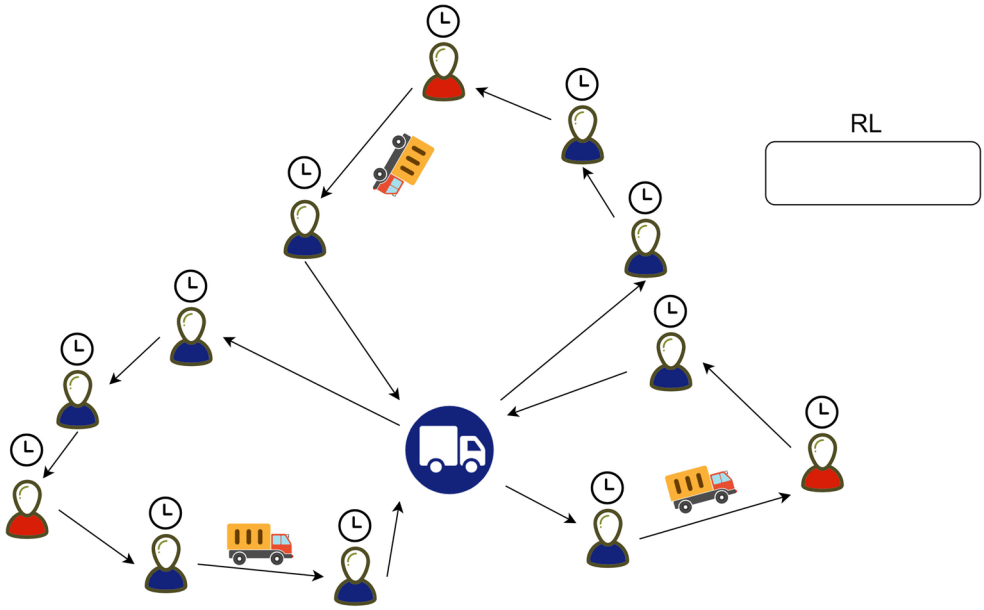

Figure 3. Repaired. According to different repair operators. The figure shows the new path generated after inserting the removed clients into the optimal position.

Cooling standards	Cooling function	Parametric
Standard cooling	$T_0 = T_0 * \alpha$	$T_0 = 200, \alpha = 0.99$
Fast cooling	$T = T_0 * \exp(-\alpha * k^{1/2})$	$T_0 = 200, \alpha = 0.99$ k is the number of iterations
This article	$T_0 = T_0 * \alpha T = T_0 * (1 + \cos(t * pi/gapIter))$	$T_0 = 100, \alpha = 0.99$ $gapIter = 100$

Table 1. Cooling function and parameters.

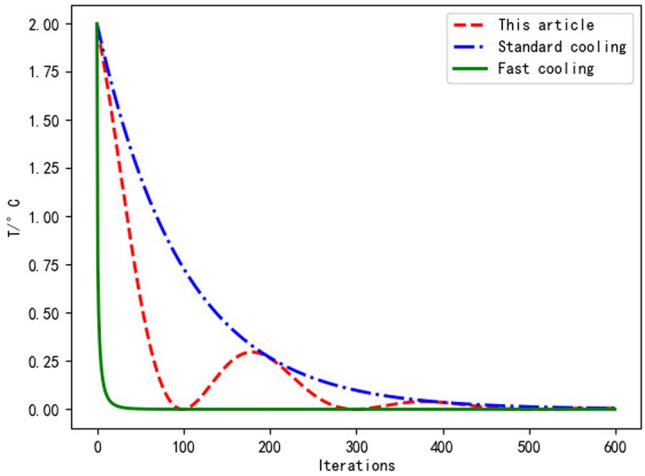

Figure 4. Comparison of cooling curves. The X-axis is the number of iterations, and the Y-axis is the current temperature. The red curve is the cooling curve of This article, the blue curve is the cooling curve of Standard cooling, and the green curve is the cooling curve of Fast cooling.

Random-criticality repair

Optimizing for the total transport cost without exceeding the transport vehicle's maximum load capacity, the order of importance of all customers in the *RL* is determined according to Eq. (12). The customer with high importance is inserted first, and the customer is inserted at a random position in the random path, and then the customer is removed from the *RL*. If there is no customer in the *RL* that needs the service, then the repair phase is exited, otherwise, the study continues with the insertion of the next customer in the order of importance.

$$\varphi_i = \frac{\eta(d_i)}{\eta(b_i - a_i)} + \eta(c_{0i}) \tag{12}$$

φ_i represents the importance of customer point i, η is a normalized function, d_i represents the demand of customer point i, $(b_i\text{-}a_i)$ represents the service time window of customer point i and c_{0i} represents the distance of customer point i from the distribution center.

Improvement of the cooling function

The main components of the simulated annealing algorithm, including the state generating function, the state acceptance function, the cooling function, the thermal equilibrium stability criterion, the annealing end criterion, and the selection of the initial temperature. Among them, the cooling function affects the algorithm to collect the search space. If the temperature is high, the algorithm carries out the global search. In the contrary, if the temperature is low, the algorithm carries out the local search. If the global search time of the Temperature cooling faster algorithm is too short, it can't get the global optimal solution. At the same time, if the local search ability of the cooling too slow algorithm is weaker, it can affect the quality of the solution. Both of them will make the algorithm reduce the quality of the solution. Therefore, the cooling function of the algorithm is improved, and the improved cooling function is shown in Eqs. (13) and (14):

$$T_0 = T_0 \times \alpha \tag{13}$$

$$T = T_0 \times (1 + \cos(t \times pi/gapIter)) \tag{14}$$

In Eqs. (13) and (14), T_0 denotes the initial temperature, T denotes the temperature after the change, α is the coefficient of cooling, t represents the number of iterations, and *gapIter* denotes the number of intervals for floating.

Standard cooling[41], Fast cooling[42] ,and the cooling function of this paper were selected to compare the effect. The initial conditions, cooling function, and parameters are shown in Table 1, and the cooling curve is shown in Fig. 4.

As shown in Fig. 4, the standard cooling reaches the low-temperature state with too many iterations, while the fast cooling is too fast, which is easy to fall into the local optimal solution. In this paper, the cooling combines the advantages of both, while increasing the tempering process so that the algorithm's global search and local search ability will be strengthened.

Parallel ALNS process

A parallel strategy is proposed to increase the search space and reduce the running time of ALNS. The Greedy Insertion operator has the advantage that it can find the local optimal solution, but it takes the longest time because it has to try all the possibilities. In this paper, when the Greedy Insertion operator is chosen, the Master will Shuffle according to the vehicles, and the Workers will pass the total transport cost after location insertion to each other to finally determine the repair order. When other operators are chosen, several populations are deposited into the RDD, and the perturbations are run in parallel to generate new solutions.

Spark-ALNS steps are as follows:

Step 1: Initialize the population according to the VRPTW instance CW, which is stored in RDD.
Step 2: Calculate the fitness value and record the optimal path P_{best} and optimal distance L_{best}.
Step 3: Roulette chooses to destroy operators, and repair operators determine whether the repair operators are greedy insertions. If yes, Shuffle according to vehicle. If no, Shuffle according to population. Calculate the fitness value and select the current optimal path P_{new} in the population.
Step 4: Update the temperature according to Eqs. (13) and (14), and compare the difference between the fitness values of the two paths, if the $f(P_{new}) < f(P_{best})$, then $P_{best}=P_{new}$. Otherwise, update according to the Metropolis criterion.
Step 5: Determine if the end condition has been reached. If it is not reached, then execute the Step3 to Step5, otherwise output the result.

In summary, the flowchart of the algorithm is shown in Fig. 5.

Computational results

In order to test the performance of the algorithm Spark-ALNS in this paper, the standard dataset in Solomon is used as the experimental data for testing. In our proposed Spark-ALNS method, the irace package is used to determine the parameters, and the main parameters are shown in Table 2.

Solving for quality comparisons

This paper compares with HGA-SIH[43]and ACS-BSO[44] to verify the effectiveness of the algorithms. The results are shown in Table 3, where Best is the known optimal solution, VN is the number of vehicles used, and DIS is the distribution distance. The black font in Table 3 indicates the optimal results, and the ones with * are better than optimal solution dataset results. In Table 3, for problem sets C1 and C2, the partial solutions of HGA-SIH and ACS-BSO are not optimal, while all instances of Spark-ALNS find optimal solutions, and 2 instances found new optimal solutions. For problem sets R1 and R2, the quality of solutions of HGA-SIH and ACS-BSO in R2 is significantly lower than that of Spark-ALNS. For problem sets RC1 and RC2, HGA-SIH and ACS-BSO each have 2 instances with optimal solutions, Spark-ALNS has 10 instances with optimal solutions, and 7 instances found new optimal solutions. Therefore, it can be concluded that Spark-ALNS solves VRPTW better. The partially optimal path is shown in Fig. 6.

Parallel versus serial algorithms

In this paper, in order to verify the advantages of parallel algorithms over serial algorithms, six kinds of instances were used to carry out serial and parallel operations, parallel computing using two processors and three processors for comparison. The results are shown in Table 4. To more intuitively react to the number of processors and parallel algorithms on the results of the impact of the calculation time of the drawing of the bar graph as shown in Fig. 7. The experimental data shows that using the parallel framework for computation, two processors are going to save about 80% of the time than the serial algorithm under the same conditions, and three processors are going to save about 80-90% of the time than the serial algorithm under the same conditions. But the computation time of three processors is about 0.6 times of the running time of two processors. It can be seen that Spark-ALNS is much more computationally efficient than the traditional serial framework, and the impact of the increase in the number of machines on the reduction of the computation time is very obvious by increasing the number of processors by a certain amount of time, it is possible to reduce the computation time and to ensure or even improve the quality of the final solution.

Parallel algorithm experimental

In this section, the three parallel algorithms ACS[45], CPT[46]and PMA[47] are compared with each other by conducting 10 independent experiments, comparing the distance DIS and the time T(s), respectively. And the obtained data are shown in Table 5, where the convergence process of R204 and RC204 are compared in Figs. 8 and 9. As shown in Table 5, Spark-ALNS obtains better solutions in different instances, especially in the case of the problem set RC, the total distance of the solution is significantly better than the other three algorithms. Spark-ALNS solves the problem sets C and R, the optimal solution can be obtained in 10s, while the other three algorithms take 5s to 45s to obtain the optimal solution. Spark-ALNS can find the optimal solution in 10s to 40s

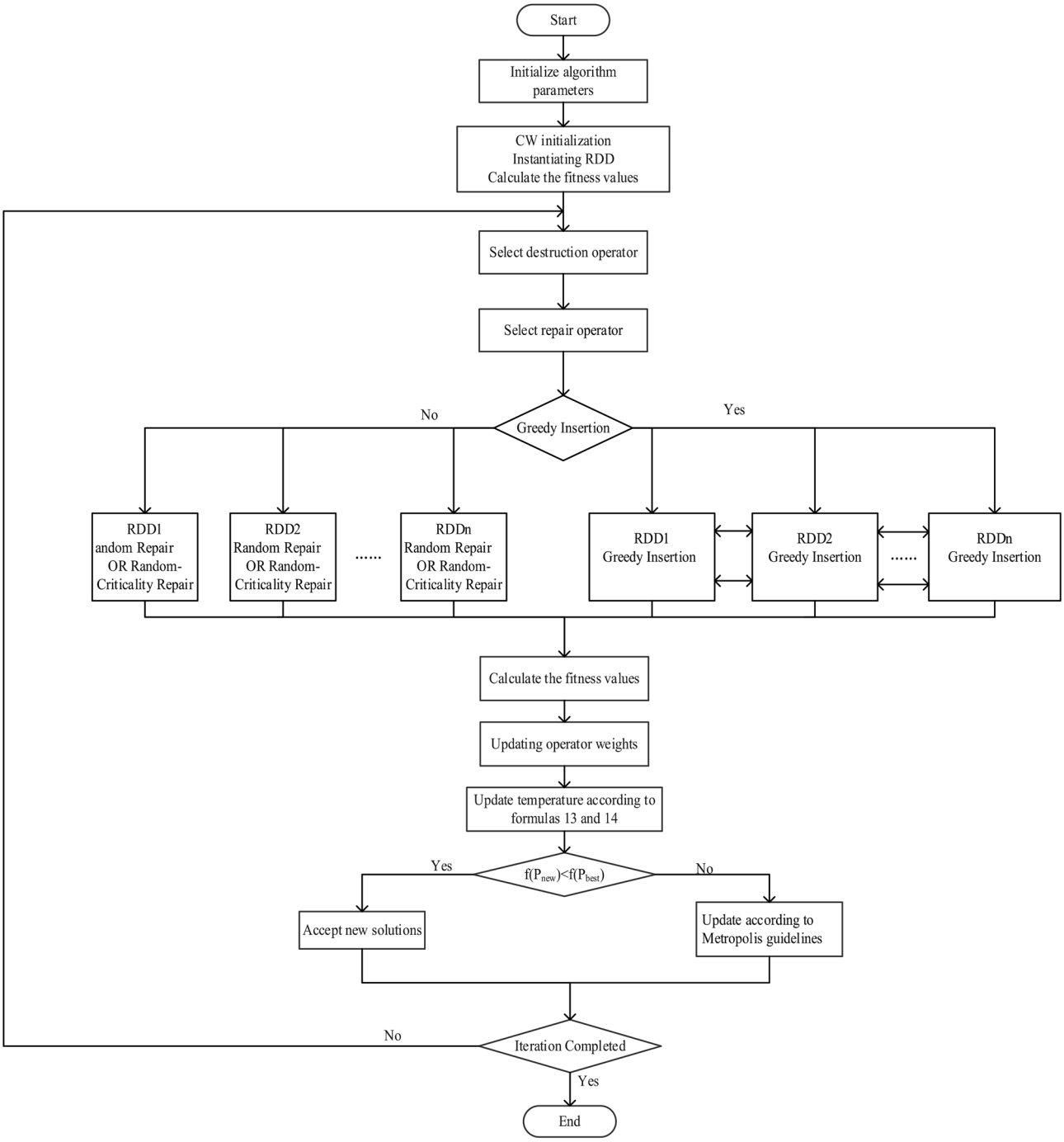

Figure 5. Spark-ALNS algorithm flow chart.

Parameters category	Parameters	Hidden meaning	Value
SA related parameters	T	Initial temperature	100
	α	Cooling factor	0.95
	gapIter	Floating intervals	55
Operator weights update parameters	t	Number of update iterations	60
	r	Updating parameters	0.65
Operator scoring parameters	W_1	Score of the new global optimal solution	35–50
	W_2	Score for the first occurrence of a feasible solution	15–25
	W_3	Score for first occurrence of infeasible solutions	5–10

Table 2. Related parameters of spark-ALNS.

Instances	Best		HGA-SIH		ACS-BSO		Spark-ALNS	
	VN	DIS	VN	DIS	VN	DIS	VN	DIS
C101	10	828.9	10	**828.9**	10	**828.9**	10	**828.9**
C102	10	828.9	10	**828.9**	10	**828.9**	10	**828.9**
C103	10	828	10	830.7	10	**828**	10	**828**
C104	10	824.7	10	864.2	10	828.7	12	**822.9***
C105	10	828.9	10	**828.9**	10	**824.9**	10	**828.7**
C106	10	828.9	10	**828.9**	10	**828.9**	10	**828.9**
C107	10	828.9	10	**828.9**	10	**828.9**	10	**828.9**
C108	10	828.9	10	854.3	10	**828.9**	10	**828.9**
C109	10	828.9	10	854.7	10	**828.9**	10	**828.9**
C201	3	591.5	3	**591.5**	3	**591.5**	3	**591.5**
C202	3	591.5	3	**591.5**	3	**591.5**	3	**591.5**
C203	3	591.1	3	**591.1**	3	**591.1**	3	**591.1**
C204	3	590.5	3	594.5	3	**590.5**	3	**588.1***
C205	3	588.8	3	**588.8**	3	**588.8**	3	**588.8**
C206	3	588.4	3	**588.4**	3	**588.4**	3	**588.4**
C207	3	588.2	3	**588.2**	3	**588.2**	3	**588.2**
C208	3	588.3	3	**588.3**	3	**588.3**	3	**588.3**
R101	19	1650.7	19	1656.5	20	**1642.8**	18	**1612.2***
R102	18	1466.6	18	1476.8	18	1479.5	18	1478.6
R103	13	1292.6	14	**1230.0***	15	**1225.3***	14	**1227.6***
R104	9	1007.3	10	1010.5	12	**1002.6**	11	**996.95**
R105	14	1377.1	14	1395.6	15	**1365.6***	14	**1377.9**
R106	12	1252	13	1261.6	13	**1249.5***	12	**1252**
R107	10	1104.6	11	1103.7	12	**1091.2***	10	**986.1***
R108	9	960.8	10	966.9	10	**960.8**	9	**960.8**
R109	11	1194.7	12	1200.2	12	**1165.7***	11	**1194.7**
R110	10	1118.8	11	1127.2	11	**1090.9***	11	**1080.3***
R111	10	1096.7	11	1096.3	12	**1063.6***	10	**1096.7**
R112	9	982.1	9	**982.1**	11	976.2	9	**982.1**
R201	8	1143.2	4	1442.2	8	1161.2	7	1191.3
R202	8	1034.4	4	1212.4	6	1058.8	7	1047.4
R203	6	874.8	3	1197.6	5	883.4	6	883.2
R204	5	735.8	10	854.3	4	756.9	5	748.7
R205	5	954.1	4	1337.6	6	978.4	5	**954.1**
R206	4	879.8	4	1114.7	4	906.2	3	906.1
R207	4	797.9	4	1055.7	4	812.3	4	807.7
R208	4	705.3	3	860.5	3	725	4	**705.3**
R209	5	859.3	2	895.4	5	879	5	**859.3**
R210	6	905.2	2	942.9	7	923.4	3	939.4
R211	4	753.1	4	1049.6	4	776.1	4	**753.1**
RC101	15	1623.5	16	1656.5	16	1643.7	15	**1619.8***
RC102	14	1461.2	13	1532.2	14	1464.6	13	**1447.1**
RC103	11	1261.6	12	1344	11	1275.6	11	**1261.6**
RC104	10	1135.4	11	1184.9	10	1156.9	10	1147.2
RC105	13	1629.4	14	1662	16	**1535.7***	15	**1621.8***
RC106	13	1371.6	12	**1344.0***	13	1378.4	12	**1346.4***
RC107	12	1212.8	12	1263	12	1216.6	12	**1207.8***
RC108	11	1117.5	11	1144.9	11	1134.2	11	**1114.2***
RC201	9	1265.5	4	1442.2	8	1284.7	9	1275.3
RC202	8	1095.6	4	1212.4	7	1127	8	**1092.3***
RC203	5	928.5	3	1197.6	6	943.1	6	950
RC204	4	786.3	10	854.3	4	807.7	3	797.9
RC205	7	1157.5	4	1337.6	8	1170.9	7	**1157.5**
RC206	7	1054.6	4	1114.7	7	1093.6	5	1089.2
RC207	3	1061.1	4	**1055.7***	6	**986.7***	6	**978.9***
RC208	4	779.3	3	860.53	4	785.6	4	783.1

Table 3. Experimental comparison results.

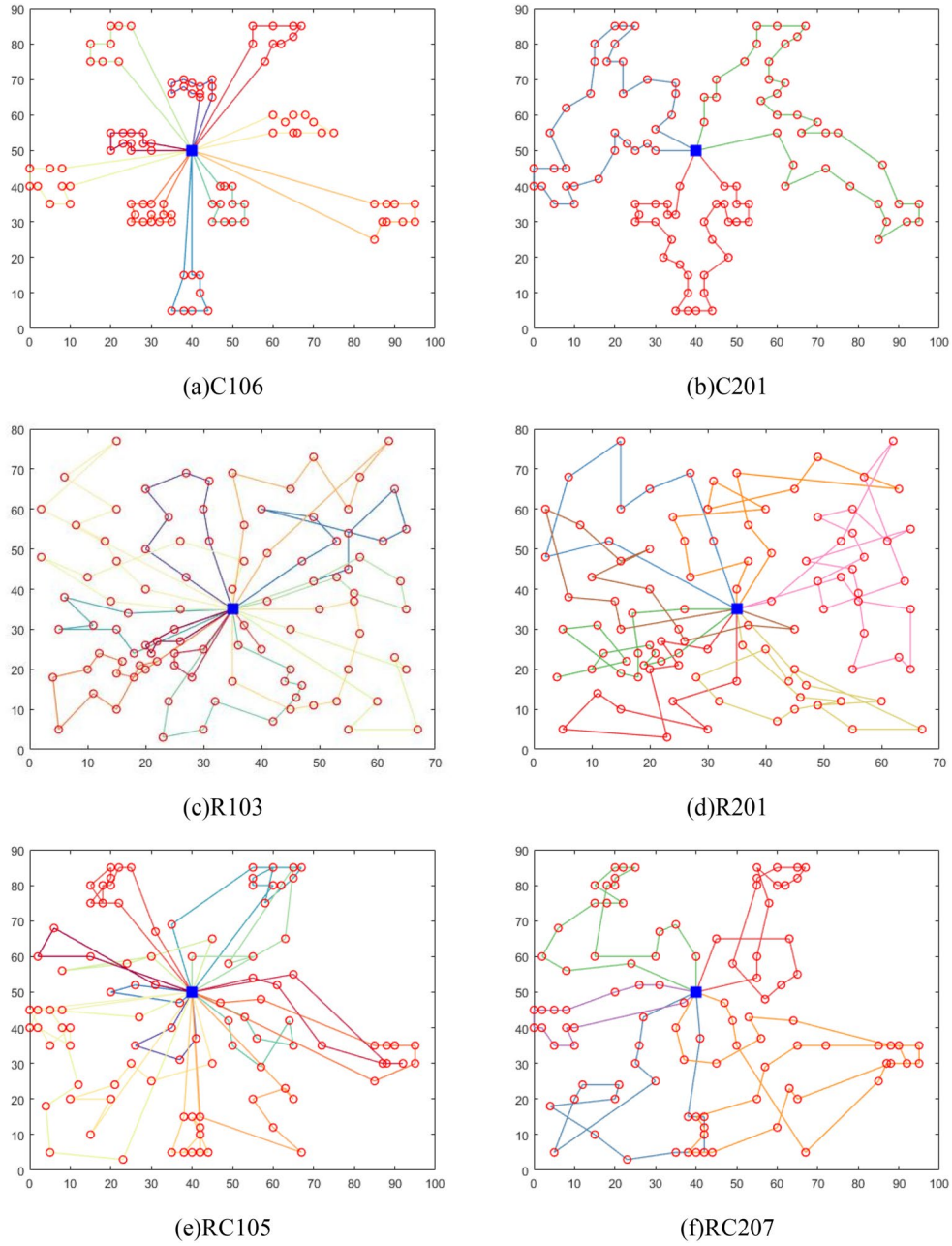

Figure 6. Schematic representation of the best solutions found by Spark-ALNS on 6 instances, X-axis and Y-axis, is the position of the coordinate points of the customer and distribution center. Different colors are the distribution paths of different distribution vehicles.

instances	Three processor runtime(s)	Two processor runtime(s)	Serial Runtime(s)
C101	19.3	31.4	157.9
C201	26.8	45.2	283.5
R101	52.6	60.8	358.3
R201	65.3	82.3	432.8
RC101	64.7	87.9	473.4
RC201	81.9	96.3	536.3

Table 4. Parallel versus serial algorithms results.

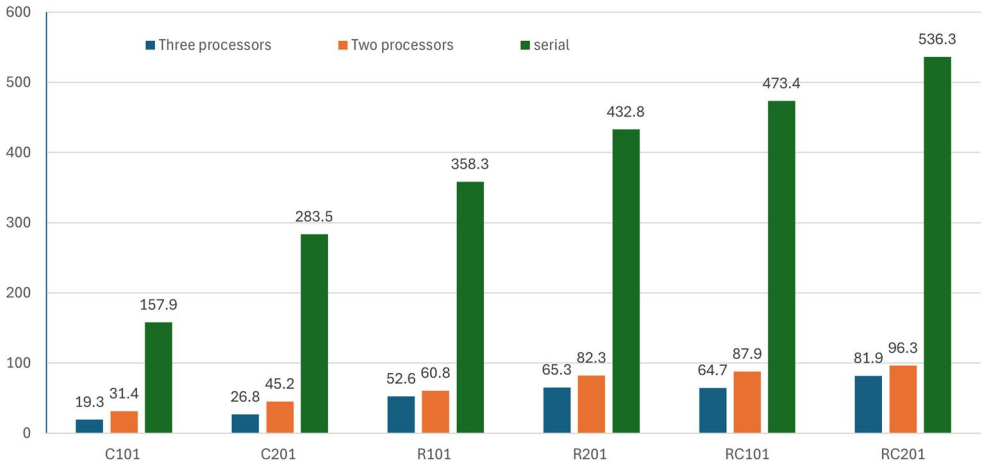

Figure 7. Serial and parallel algorithm runtimes. Blue is the three-processor runtime, orange is the two-processor runtime, and green is the serial runtime.

Instances	ACS		CPTS		PMA		Spark-ALNS	
	DIS	T(s)	DIS	T(s)	DIS	T(s)	DIS	T(s)
C104	824.7	7.6	824.7	5.1	824.7	6.3	822.9	1.2
C203	591.1	13.8	595.2	17.5	596.1	21.8	591.1	4.8
R106	1350.9	17.8	1369.3	23.7	1382.7	28.1	1252	3.5
R204	758	15.6	759.6	26.3	779	30.7	748.7	7.9
RC103	1388.1	34.2	1410.2	41.7	1438.2	52.3	1261.6	13.7
RC204	821.7	50.5	830.2	67.4	845.6	75.1	797.9	21.5

Table 5. Parallel algorithm experimental results.

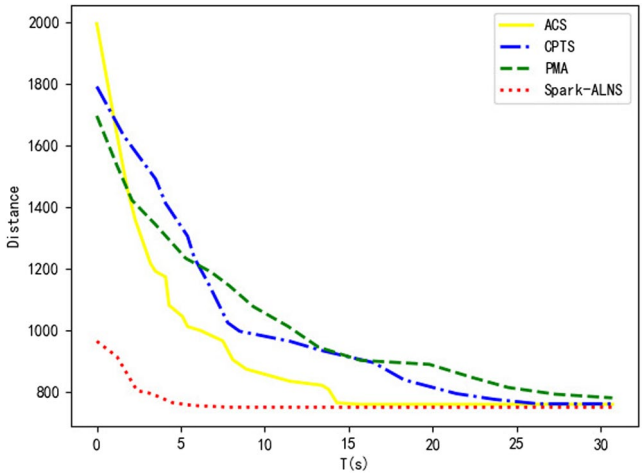

Figure 8. Comparison of R204 data convergence curves. The X-axis is the running time in s, and the Y-axis is the distribution distance. The yellow curve is ACS, the blue curve is CPTS, the green curve is PMA, and the red curve is Spark-ALNS.

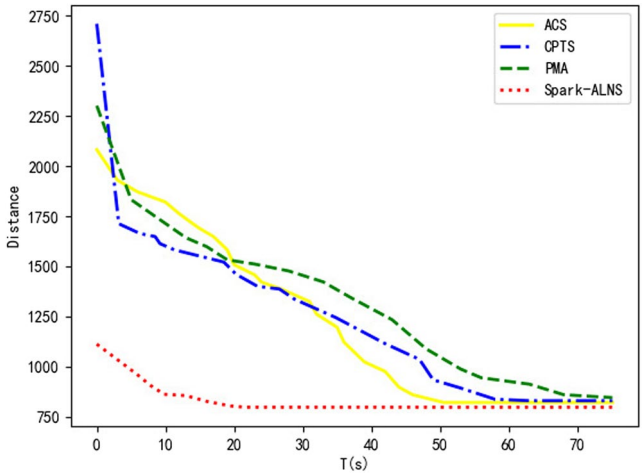

Figure 9. Comparison of RC204 data convergence curves. The X-axis is the runtime in s. The Y-axis is the distribution distance. The yellow curve is ACS, the blue curve is CPTS, the green curve is PMA, and the red curve is Spark-ALNS.

in solving the problem set RC, while the other three algorithms need 30s to 100s to find the optimal solution, and the convergence time is 3–5 times faster than the other three algorithms. As can be seen in Figs. 8 and 9, the CW initialization makes the initial solution of Spark-ALNS better than the other three parallel algorithms, including the convergence time is more faster, and the solution quality is more better.

Conclusion

With the rapid development of the transport and logistics distribution industry, VRPTW has arisen, and it is an important branch derived from VRP. In this paper, we solve VRPTW by parallel ALNS algorithm based on Spark framework, modifying the cooling function, adopting the adaptive neighborhood selection operator strategy with three destruction operators and three repair operators, and focusing on reducing the problem of large time complexity of ALNS on Greedy Insertion on the parallel framework. The problem of a large number of neighbors and lack of guidance in neighborhood selection during the search process is solved, which enhances the local optimality search capability, reduces the running time, and increases the solution quality. The experimental results show that known optimal solutions are obtained in 41 out of 56 instances, and new optimal solutions are found for 31 algorithms. This paper also compares the impact of serial and parallel algorithms and demonstrates the effectiveness of Spark-ALNS and the impact of multiple workers on the runtime. In addition, the algorithm in this paper is compared with other parallel algorithms, and Spark-ALNS outperforms other algorithms, effectively improving the solution quality and converging faster.

This study mainly considered the transport vehicle scheduling problem in a single warehouse, but in real application scenarios such as courier services and shuttle bus pickups, the multilevel multi-warehouse problem is equally prevalent, and the applicability of the algorithms to the multilevel problem deserves further study. The next study can work with other algorithms to achieve better results.

Data availability

The datasets analysed during the current study are available in the Solomon repository, http://web.cba.neu.edu/~msolomon/problems.htm.

Received: 20 June 2024; Accepted: 26 September 2024
Published online: 11 October 2024

References

1. Othman, W., Wahab, A., Alhady, S. & Wong, H. N. Solving vehicle routing problem using ant colony optimization (ACO) algorithm. *Int. J. Res. Eng.***5**, 500–507 (2018).
2. Ibrahim, M. F., Putri, M., Farista, D. & Utama, D. M. An improved genetic algorithm for vehicle routing problem pick-up and delivery with time windows. *Jurnal Teknik Industri*. **22**, 1–17 (2021).
3. Zhu, B. et al. A critical scenario search method for intelligent vehicle testing based on the social cognitive optimization algorithm. *IEEE Trans. Intell. Transp. Syst.***24**, 7974–7986 (2023).
4. Zhao, J. et al. A human-like trajectory planning method on a curve based on the driver preview mechanism. *IEEE Trans. Intell. Transp. Syst.***24**, 11682–11698 (2023).
5. Liu, X. et al. Trajectory prediction of preceding target vehicles based on lane crossing and final points generation model considering driving styles. *IEEE Trans. Veh. Technol.***70**, 8720–8730 (2021).
6. Solomon, M. M. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper. Res.***35**, 254–265 (1987).
7. Corstjens, J., Depaire, B., Caris, A. & Sörensen, K. A multilevel evaluation method for heuristics with an application to the VRPTW. *Int. Trans. Oper. Res.***27**, 168–196 (2020).

8. Aggarwal, D. & Kumar, V. Performance evaluation of distance metrics on firefly algorithm for VRP with time windows. *Int. J. Inform. Technol.***13**, 2355–2362 (2021).
9. Hosseinabadi, A. A. R., Rostami, N. S. H., Kardgar, M., Mirkamali, S. & Abraham, A. A new efficient approach for solving the capacitated vehicle routing problem using the gravitational emulation local search algorithm. *Appl. Math. Model.***49**, 663–679 (2017).
10. Hosseinabadi, A. A. R., Alavipour, F., Shamshirbnd, S. & Balas, V. E. A novel meta-heuristic combinatory method for solving capacitated vehicle location-routing problem with hard time windows. In: Information Technology and Intelligent Transportation Systems: Volume 1, Proceedings of the 2015 International Conference on Information Technology and Intelligent Transportation Systems ITITS 2015, held December 12–13, 2015, Xi'an China). Springer (2017).
11. Nategh, M. N., Hosseinabadi, A. A. R. & Balas, V. E. Ant_VRP: ant-colony-based meta-heuristic algorithm to solve the vehicle routing problem. *Int. J. Adv. Intell. Paradigms*. **11**, 315–334 (2018).
12. Tirkolaee, E. B., Hosseinabadi, A. A. R., Soltani, M., Sangaiah, A. K. & Wang, J. A hybrid genetic algorithm for multi-trip green capacitated arc routing problem in the scope of urban services. *Sustainability*. **10**, 1366 (2018).
13. Tirkolaee, E. B., Alinaghian, M., Hosseinabadi, A. A. R., Sasi, M. B. & Sangaiah, A. K. An improved ant colony optimization for the multi-trip Capacitated Arc Routing Problem. *Comput. Electr. Eng.***77**, 457–470 (2019).
14. Gocken, T. & Yaktubay, M. Comparison of different clustering algorithms via genetic algorithm for VRPTW. (2019).
15. Olaniyi, O. S., James, A. K., Ibrahim, A. A. & Makanjuola, A. F. On the application of a modified genetic algorithm for solving vehicle routing problems with time windows and split delivery. *IAENG Int. J. Appl. Math.***52**, 1–9 (2022).
16. Pirozmand, P. et al. *D-PFA: A Discrete Metaheuristic Method for Solving Traveling Salesman Problem Using Pathfinder Algorithm* (IEEE Access, 2023).
17. Tadaros, M. & Kyriakakis, N. A. A hybrid clustered ant colony optimization Approach for the hierarchical Multi-switch Multi-echelon Vehicle Routing Problem with Service Times. *Comput. Ind. Eng.***190**, 110040 (2024).
18. Kumar, S. et al. A two-archive multi-objective multi-verse optimizer for truss design. *Knowl. Based Syst.***270**, 110529 (2023).
19. Mashru, N., Tejani, G. G., Patel, P. & Khishe, M. Optimal truss design with MOHO: a multi-objective optimization perspective. *Plos One*. **19**, e0308474 (2024).
20. Kumar, S. et al. *Optimization of Truss Structures Using multi-objective Cheetah Optimizer*1–22 (Mechanics Based Design of Structures and Machines, 2024).
21. Focacci, F., Laburthe, F. & Lodi, A. Local search and constraint programming. In: Handbook of Metaheuristics). Springer (2003).
22. Ropke, S. & Pisinger, D. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transport. Sci.***40**, 455–472 (2006).
23. Aydınalp, Z. & Özgen, D. Solving vehicle routing problem with time windows using metaheuristic approaches. *Int. J. Intell. Comput. Cybernetics*. **16**, 121–138 (2023).
24. Dumez, D., Lehuédé, F. & Péton, O. A large neighborhood search approach to the vehicle routing problem with delivery options. *Transp. Res. Part. B: Methodological*. **144**, 103–132 (2021).
25. Zeebaree, S. R. et al. Characteristics and analysis of hadoop distributed systems. *Technol. Rep. Kansai Univ.***62**, 1555–1564 (2020).
26. Esmaeilzadeh, A. et al. Efficient large scale nlp feature engineering with apache spark. In: 2022 IEEE 12th Annual Computing and Communication and (CCWC)). IEEE (2022).
27. Ketu, S., Mishra, P. K. & Agarwal, S. Performance analysis of distributed computing frameworks for big data analytics: hadoop vs spark. *Computación Y Sistemas*. **24**, 669–686 (2020).
28. Chambers, B. & Spark, M. Z. The Definitive Guide, Big Data Processing Made Simple.). Published by O'Reilly Media (2018).
29. Tang, J., Xu, M., Fu, S. & Huang, K. A scheduling optimization technique based on reuse in spark to defend against apt attack. *Tsinghua Sci. Technol.***23**, 550–560 (2018).
30. Hidri, K. K., Bilas, A. & Kozanitis, C. HetSpark: a framework that provides heterogeneous executors to Apache Spark. *Procedia Comput. Sci.***136**, 118–127 (2018).
31. Maqbool, F., Razzaq, S., Lehmann, J. & Jabeen, H. Scalable distributed genetic algorithm using apache spark (s-ga). In: Intelligent Computing Theories and Application: 15th International Conference, ICIC 2019, Nanchang, China, August 3–6, 2019, Proceedings, Part I 15). Springer (2019).
32. Teijeiro, D. et al. Land consolidation through parcel exchange among landowners using a distributed spark-based genetic algorithm. *J. Supercomputing*. **78**, 19522–19544 (2022).
33. Alhenawi Ea, Khurma, R. A., Sharieh, A. A., Al-Adwan, O., Al Shorman, A. & Shannaq, F. Parallel ant colony optimization algorithm for finding the shortest path for mountain climbing. *IEEE Access.***11**, 6185–6196 (2023).
34. AlJame, M., Ahmad, I. & Alfailakawi, M. Apache spark implementation of whale optimization algorithm. *Cluster Comput.***23**, 2021–2034 (2020).
35. Wang, Z., Zhao, Y., Liu, Y. & Lv, C. A speculative parallel simulated annealing algorithm based on Apache Spark. *Concurr Comput: Pract Exper***30**, e4429 (2018).
36. Guo, H., Wang, J., Sun, J. & Mao, X. Multi-objective green vehicle scheduling problem considering time window and emission factors in ship block transportation. *Sci. Rep.***14**, 10796 (2024).
37. Durgut, R. & Aydin, M. E. Adaptive binary artificial bee colony algorithm. *Appl. Soft Comput.***101**, 107054 (2021).
38. Sonuç, E. & Özcan, E. An adaptive parallel evolutionary algorithm for solving the uncapacitated facility location problem. *Expert Syst. Appl.***224**, 119956 (2023).
39. Huang, M., Liu, M. & Kuang, H. Vehicle routing problem for fresh products distribution considering customer satisfaction through adaptive large neighborhood search. *Comput. Ind. Eng.***190**, 110022 (2024).
40. Clarke, G. & Wright, J. W. Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.***12**, 568–581 (1964).
41. Bertsimas, D. & Tsitsiklis, J. Simulated annealing. *Stat. Sci.***8**, 10–15 (1993).
42. Vakil-Baghmisheh, M-T. & Navarbaf, A. A modified very fast simulated annealing algorithm. In: 2008 International Symposium on Telecommunications). IEEE (2008).
43. Maroof, A., Ayvaz, B. & Naeem, K. Logistics optimization using hybrid genetic algorithm (hga): a solution to the vehicle routing problem with time windows (vrptw). *IEEE Access*. **12**, 36974–36989 (2024).
44. Shen, Y., Liu, M., Yang, J., Shi, Y. & Middendorf, M. A hybrid swarm intelligence algorithm for vehicle routing problem with time windows. *Ieee Access.***8**, 93882–93893 (2020).
45. Karouani, Y., Elhoussaine, Z. & Vision, C. Efficient spark-based framework for solving the traveling salesman problem using a distributed swarm intelligence method. In: International Conference on Intelligent Systems and (ISCV)). IEEE (2018). (2018).
46. El Fahim, H. A Cooperative Parallel Tabu Search Meta-heuristic for the Vehicle Routing Problem with Time Windows. (2022).
47. Blocho, M. & Czech, Z. J. A parallel EAX-based algorithm for minimizing the number of routes in the vehicle routing problem with time windows. In: 2012 IEEE 14th International Conference on High Performance Computing and Communication & 2012 IEEE 9th International Conference on Embedded Software and Systems). IEEE (2012).

Author contributions

S.L.: Conceptualization, methodology, validation, formal analysis, investigation, writing—original draft, writing—review and editing. J.S.: Resources, supervision, writing—review and editing. X.D.: Methodology, validation, writing—review and editing. G.L.: Writing—review and editing.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to J.S.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2024